2.8 倍性能提升：基于 CANN 实现 Kimi K3 高性能推理

作者：小智、阿强、启航

时间：2026年10月

昇腾Highlights

Kimi K3 低时延性能优化
TPOT 28ms -> 10ms

核心算子原生适配

面向KDA、MLA、MoE、AttnRes，
实现昇腾原生算子适配与融合加速

多种数据格式原生支持

支持原生MXFP8, MXFP4量化，进一步支持FP8量化，加速推理性能

MegaKernel：MegaKDA支持

基于Agent friendly范式 CANNBOT-DSL 快速开发 MegaKDA / MegaMoe Kernel，进一步提升推理性能

NpuGraphEx后端加速

针对 torch.compile 推出的高性能后端，基于 Fx 图与 AclGraph 架构，叠加多种独有特性，深度释放昇腾算力，实现了极致的图模式加速性能

Superkernel优化

扩大算子融合范围，减少算子细粒度调度开销

CANN
全链路优化

部署策略优化

按模块配置TP/EP/DP部署策略，平衡片上内存占用、通信成本和推理效率

NPU亲和优化

权重与数据布局、算子性能调优、昇腾CCU通信、算子融合支持

多流并行编排

根据Gated MLA/Latent MoE的结构特征与并行策略，编排多流调度，减少通信/Cube/Vec的串行等待

Dspark投机推理

草稿模型并行预测与主模型批量验证

ReplaySSM支持

支持Mamba Cache RelaySSM 特性，减少投机验证完整状态快照的内存占用和访存量

社区开源支持
CANN-Recipe-Infer
CANNBot-DSL
CANN-ops-transformer

CANN

https://gitcode.com/cann

目录

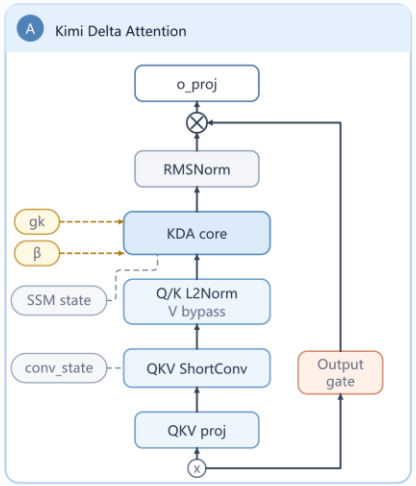
Part 1 模型解析

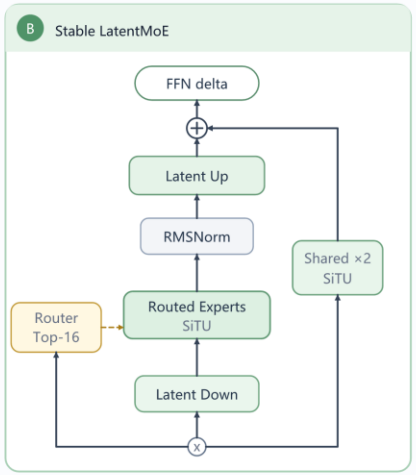
Part 2 整网优化方案解析

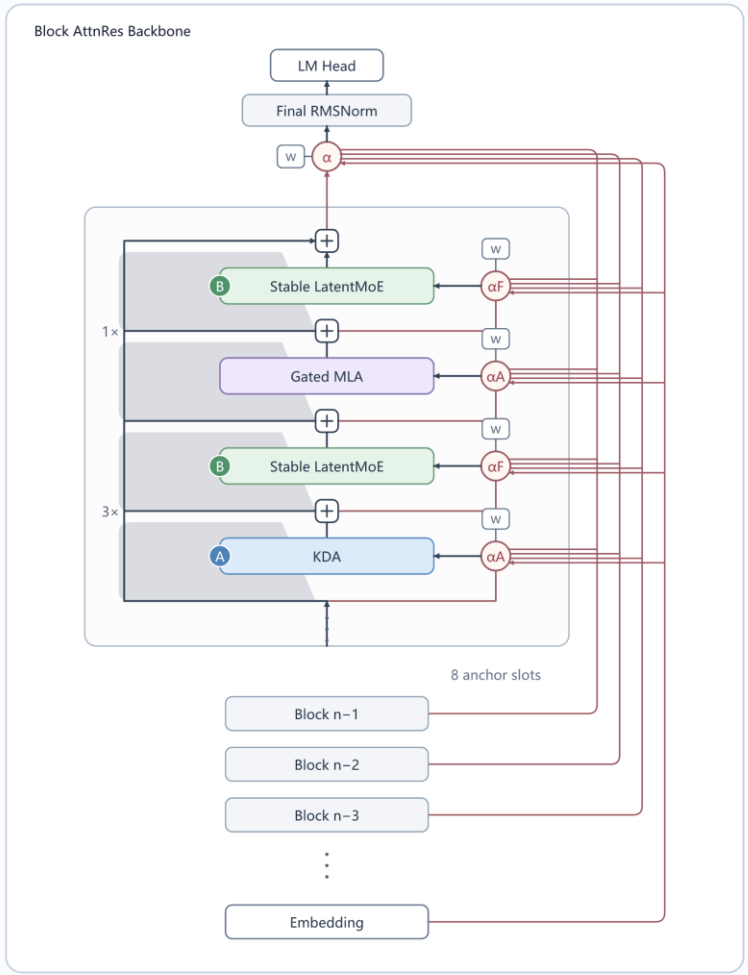
Part 3 Future Plan

Kimi-K3：整网结构与核心参数

混合注意力管理长序列，AttnRes 选择历史层表示，LatentMoE 扩展稀疏模型容量。







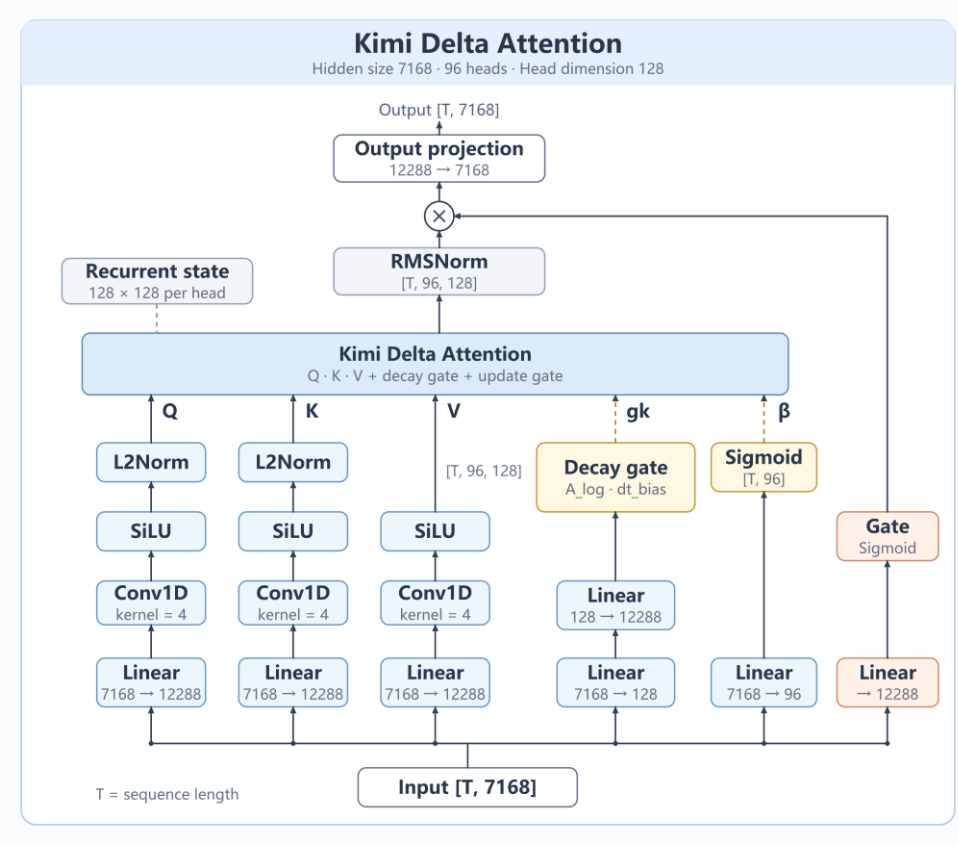
参数	Kimi K3
总参数 / 激活参数	**2.8T / 104B**
主干层数 / 隐藏维度	**93 层 / 7168**
注意力层构成	69 KDA + 24 Gated MLA
注意力头 / KDA 头维度	96 / 128
MLA 压缩秩	Q：1536；KV：512
FFN 层型	第 1 层 Dense；其余 92 层 MoE
路由专家 / 激活 / 共享	896 / Top-16 / 2
Latent / 专家中间维度	3584 / 3072
Block AttnRes	12 层/块；8 个常驻 block slots
上下文 / 词表	1M tokens / 163840
专家激活 / 量化	SiTU-GLU；MXFP4 W + MXFP8 A

混合注意力：KDA 与 MLA 协同建模长上下文

93 层 Decoder　　23 × [KDA → KDA → KDA → MLA] + 1 层 MLA　　2.8T 总参数 / 104B 激活 · 1M 上下文

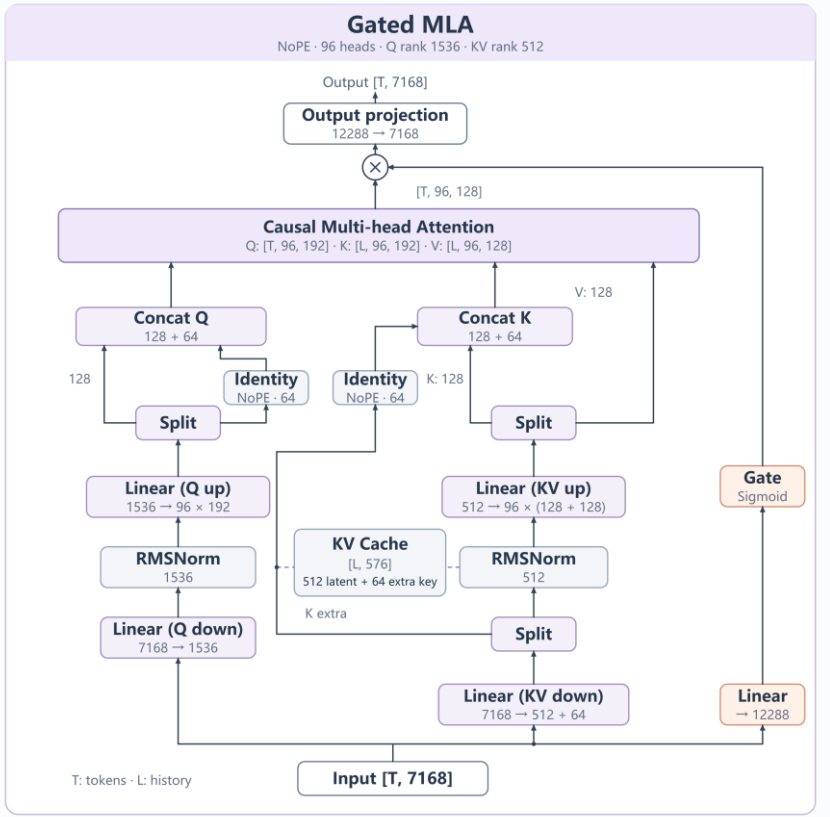
KDA｜用固定状态持续更新历史记忆

相较于GDN，从统一衰减细化为逐通道衰减，选择性保留与遗忘历史。
Delta Rule 修正记忆；每头状态为固定的 128 × 128。
69 层承担高效序列建模，状态容量不随序列长度增长。



MLA｜从压缩的 KV 历史中检索上下文

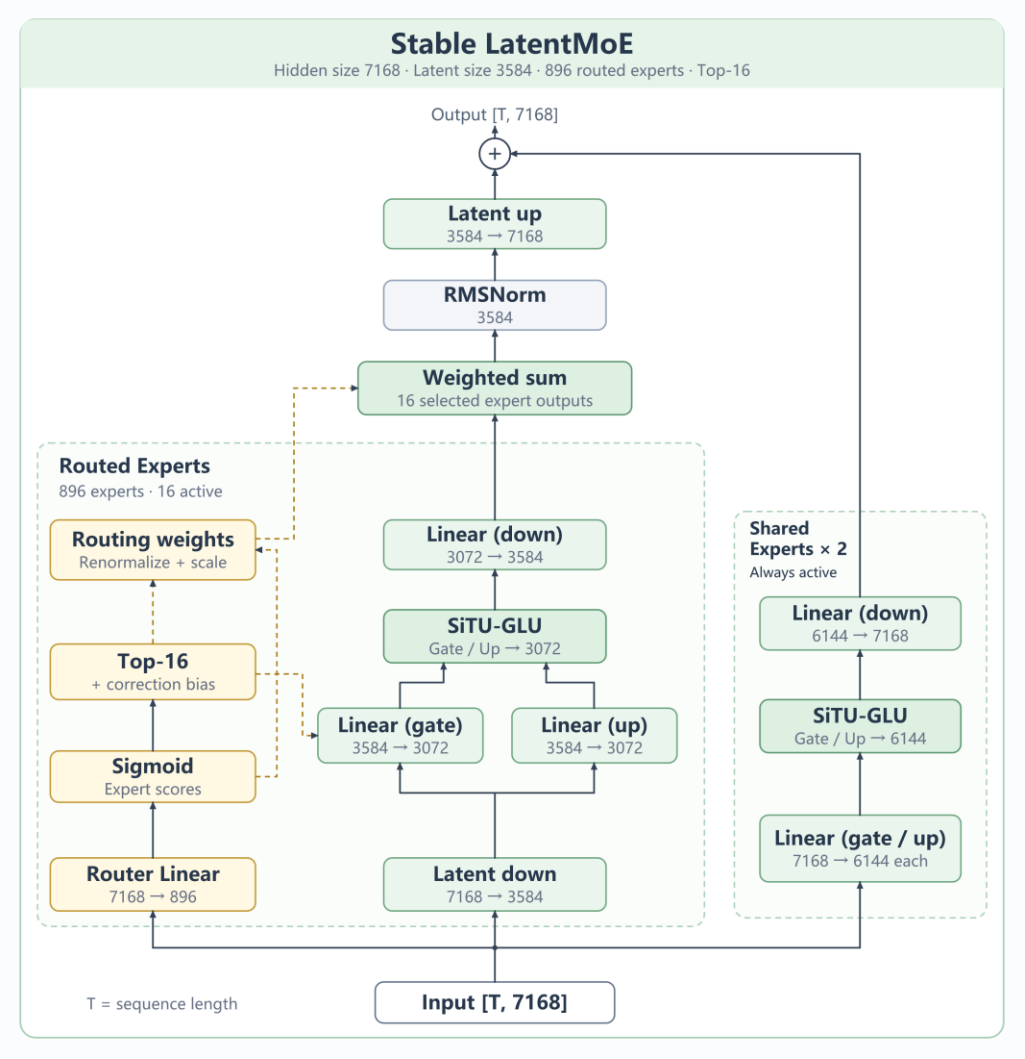
24 层、96 个头；低秩 KV 压缩支持对历史的全局检索。
每 token 缓存 576 维；输出经 Sigmoid 门控后再投影。
采用 NoPE，不施加 RoPE；与 KDA 的递推记忆互补。



Stable LatentMoE：在低维空间执行稀疏专家计算

第 2–93 层采用 Stable LatentMoE；第 1 层使用 Dense FFN。

896 路由专家 • Top-16 • 2 个共享专家

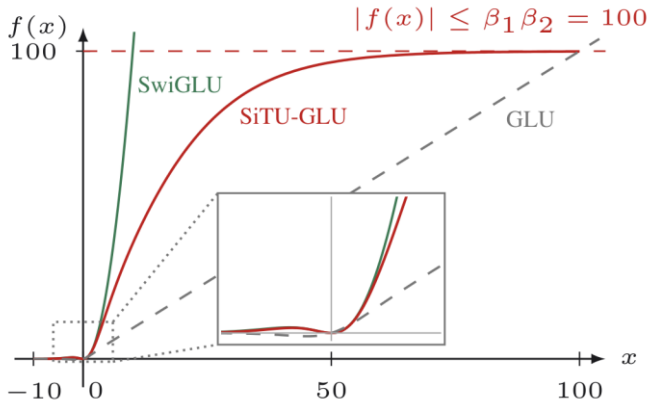


低维专家计算，兼顾容量与成本

当想激活更多 Expert（提升质量）的时候，路由的通信，读取expert两个开销都会线性放大，在进入 Expert 之前，先把 token 投影到一个更小的潜在空间 `ℓ` 里，让 Expert 在这个压缩空间里完成计算，最后再投影回来。

SiTU-GLU： softclip 抑制出现过大数值

$$\text{SiTU-GLU}(x) = [\beta_1 \tanh(W_g x / \beta_1) \odot \text{Sigmoid}(W_g x)] \odot [\beta_2 \tanh(W_u x / \beta_2)]$$



K3 报告 Figure 4（曲线局部）：两支取同一标量输入 x。

主流moe使用SwiGLU，KimiK3引入SiTU-GLU

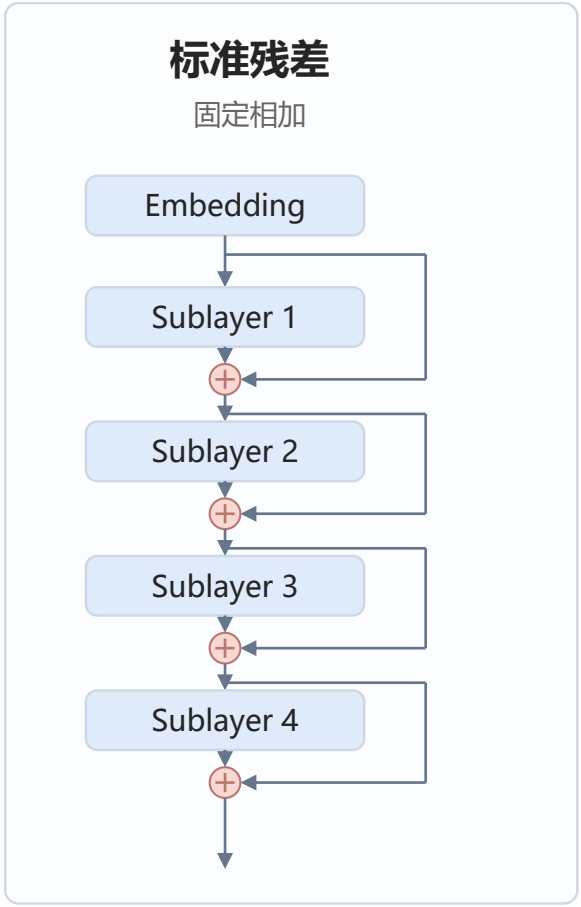
小值区接近 SwiGLU，保留局部响应。
大值区平滑饱和，限制两支乘积。
β_1=4，β_2=25；上界 100。

RMSNorm：聚合后、升维前稳定尺度

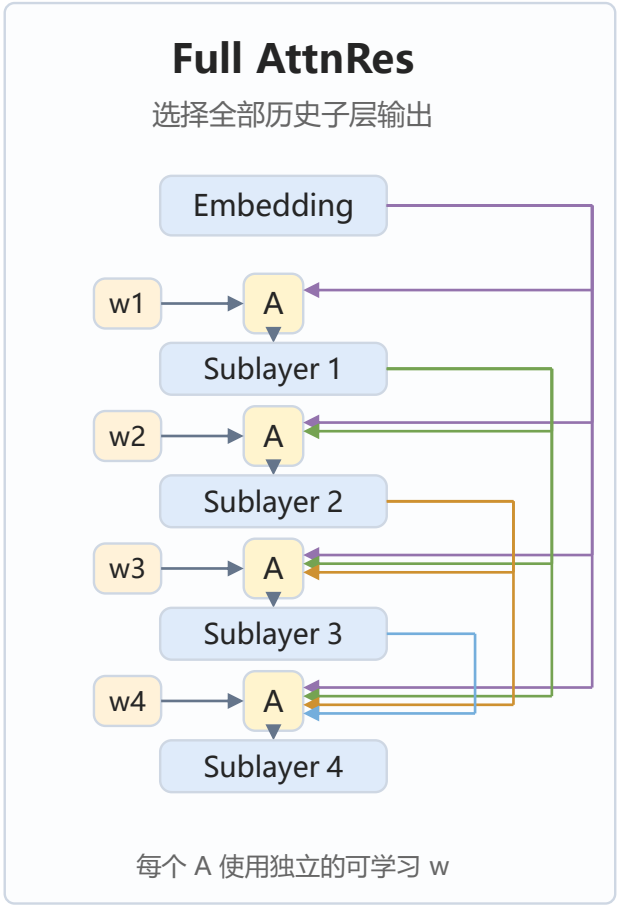
聚合 → RMSNorm（3584 维）→ Latent Up。
升维前稳定路由分支数值，再与共享分支相加。

Attention Residuals：从逐层相加到块级选择

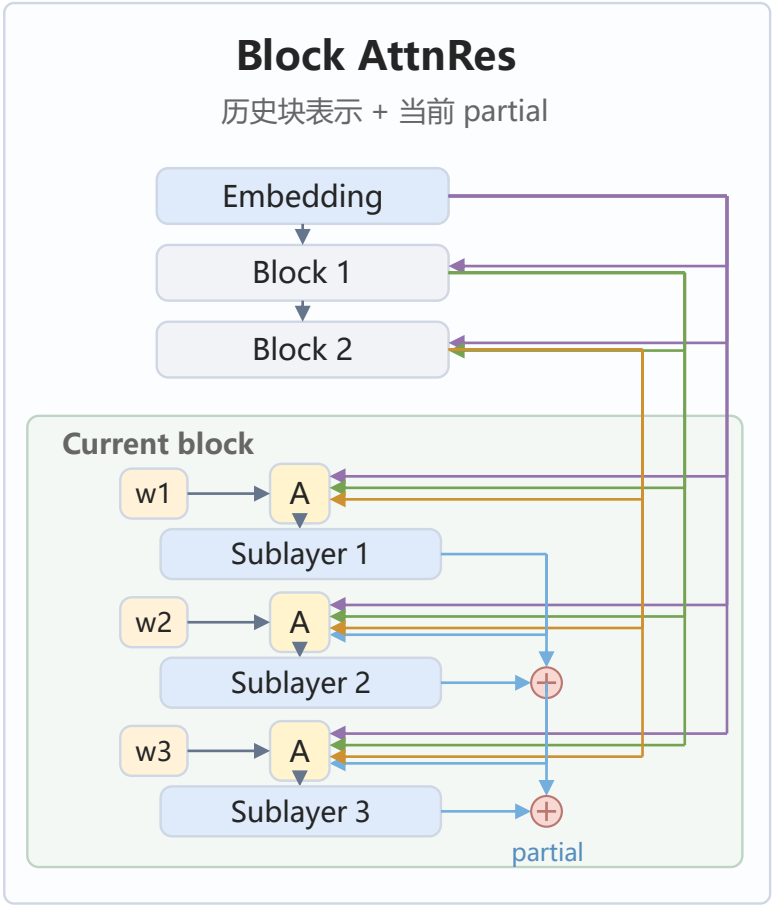
沿深度选择历史表示：Full AttnRes 细到每个子层，Block AttnRes 将历史压缩为块级候选, N=8, L=24
。



各子层输出沿残差路径累加，
历史贡献按固定方式传递。



历史候选随深度增长；
按权重直接聚合原始表示。



块内累加 partial；每个子层仍执行 AttnRes。
减少常驻历史状态，保留跨块选择能力。

目录

Part 1 模型解析

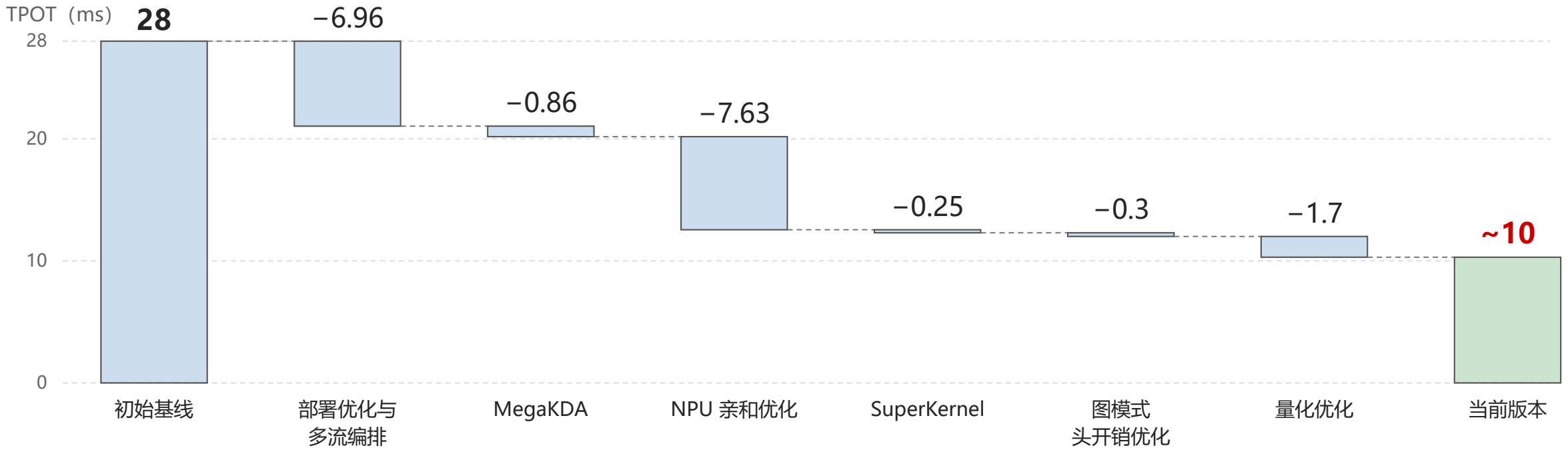
Part 2 整网优化方案解析

Part 3 Future Plan

Kimi-K3：从 28 ms 到约 10 ms

约 2.8× 加速

部署与通信、算子融合、图模式和量化共同降低每 Token 时延



部署优化与多流编排

KDA TP16 与 CCU 通信；MLA / MoE 多流重叠，减少通信与计算等待。

SuperKernel

将连续算子纳入更大的编译范围，减少调度及核启动开销。

MegaKDA

基于 CANNBot-DSL 扩大融合范围，减少中间张量读写与算子启动。

图模式头开销优化

PyTorch + CANN Aclgraph；模型级 Event 管理与跨层复用。

NPU 亲和优化

优化 NZ 布局、FA / Prolog 与通信，融合 AttnRes、SiTU + MXQuant 等计算。

量化优化

MLA W8A8C8、latent MoE W8A8；FP8 KV 数据容量相对 BF16 约减半。

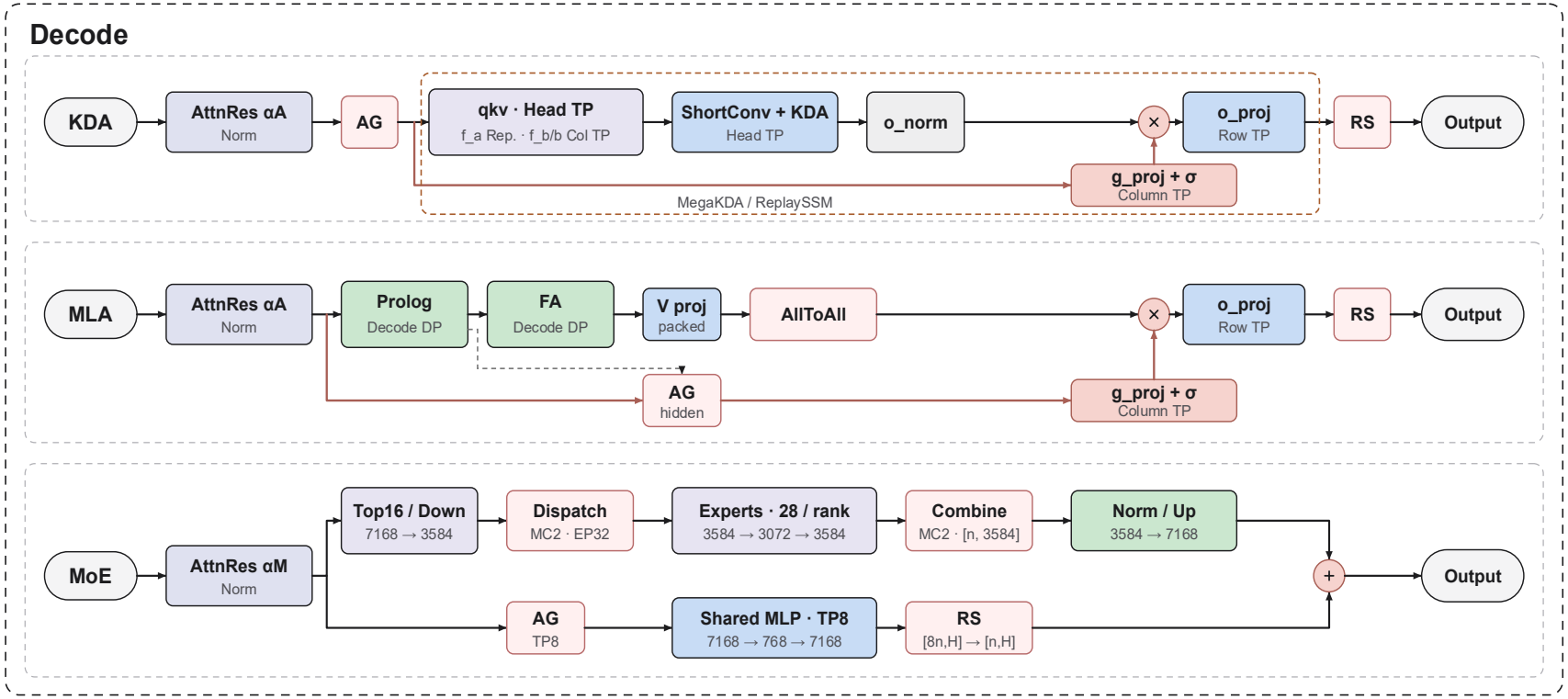
Decode 部署策略

部署策略

- 32 卡
- Attention TP16 / DP2
- Routed EP32
- Shared TP8

内存占用

模块	占用
主模型 + DSpark 主要权重	64.47 GiB
主模型 MLA KV	2.65 GiB
Prefill 独立 MLA 缓存	2.65 GiB
DSpark KV	1.96 GiB
KDA 状态及 ReplaySSM 记录	0.51 GiB
AttnRes、RoPE、固定 Mask	0.15 GiB
主要常驻小计	**72.39 GiB**
增量中间张量	1.50 GiB
通信、workspace、图内存池等	12.00 GiB
条件峰值预算	**85.89 GiB**



AG = AllGather；RS = ReduceScatter；n = 本卡 token 数；H = 7168。MLA 主路经 AllToAll 从请求 DP 转为 Head TP。

多流编排：计算通信并行，资源灵活分配

MLA | Attention 与 Gate 分支并行

Gate AllGather 与 FA 重叠；Gate 投影与 AllToAll 重叠，汇合后执行输出投影。

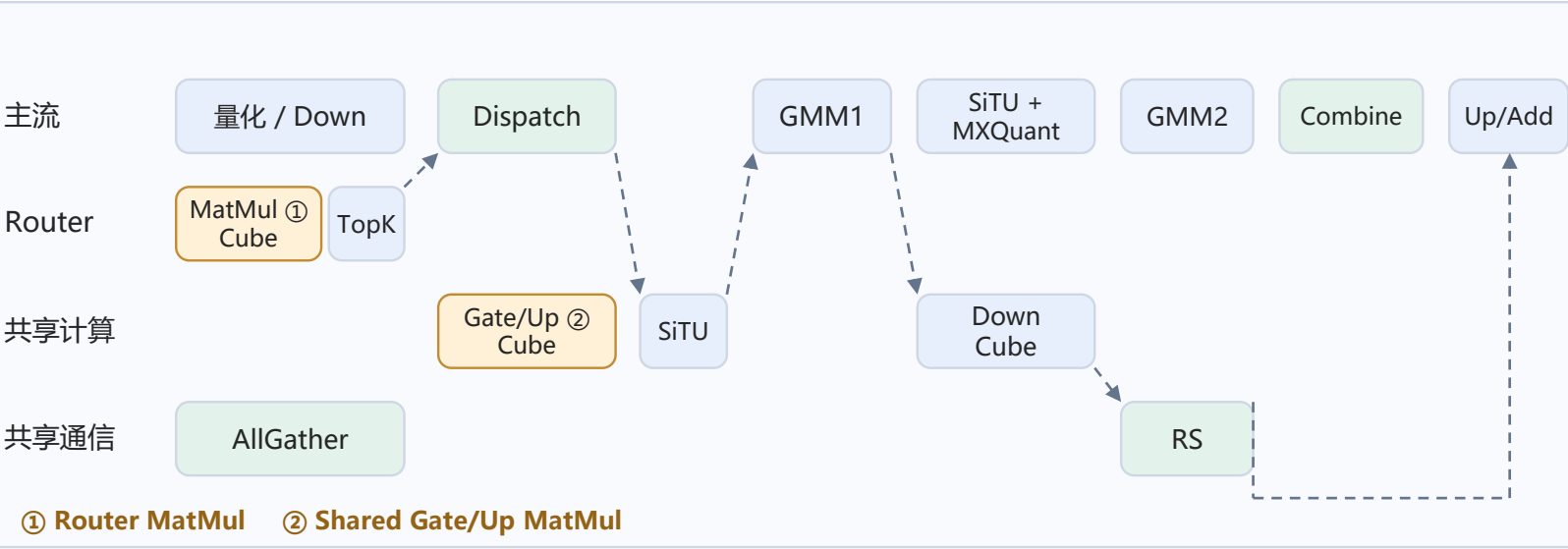


MoE | 路由、共享专家与通信并行

Router 与 Latent Down 并行，共享 AllGather 同步推进。

Dispatch 重叠 Shared Gate/Up；Shared Down 重叠路由 SiTU / 量化。

错开 Dispatch、共享 SiTU 与 GMM1，减少 Vector 资源竞争。



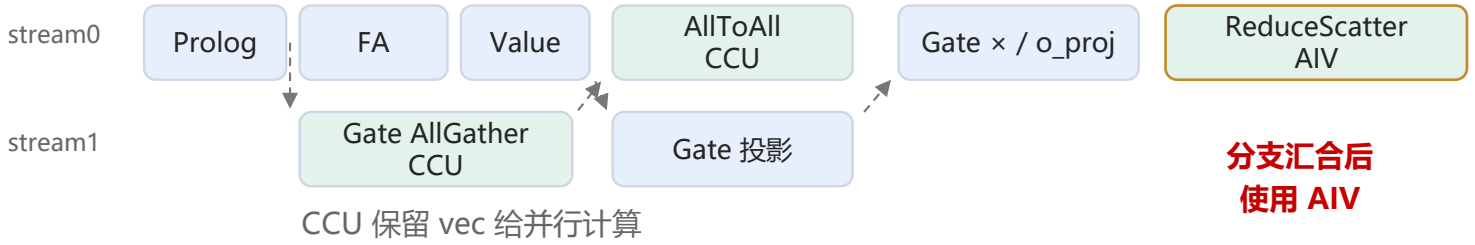
CV控核-灵活配置计算资源：算子可分配 Cube / Vector 核数，并行中减少资源抢占。

K3 实践：Router 与 Shared Gate/Up 两处 MatMul 使用 Cube，为并行的通信、激活与量化保留 Vector 资源。

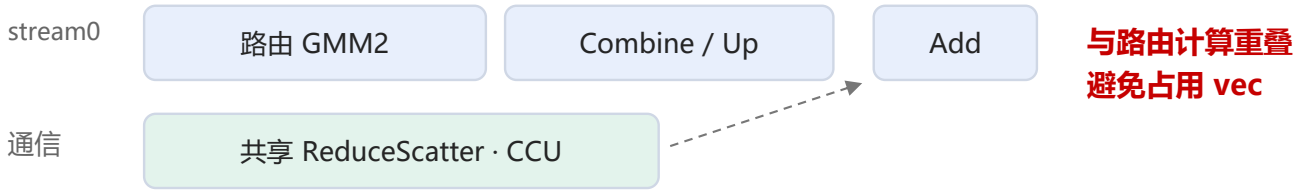
NPU 亲和优化：通信引擎、数据布局与算子调优

Kimi-K3：通信放在哪条流、用哪种引擎

MLA：CCU 与计算重叠，末尾 ReduceScatter 使用 AIV

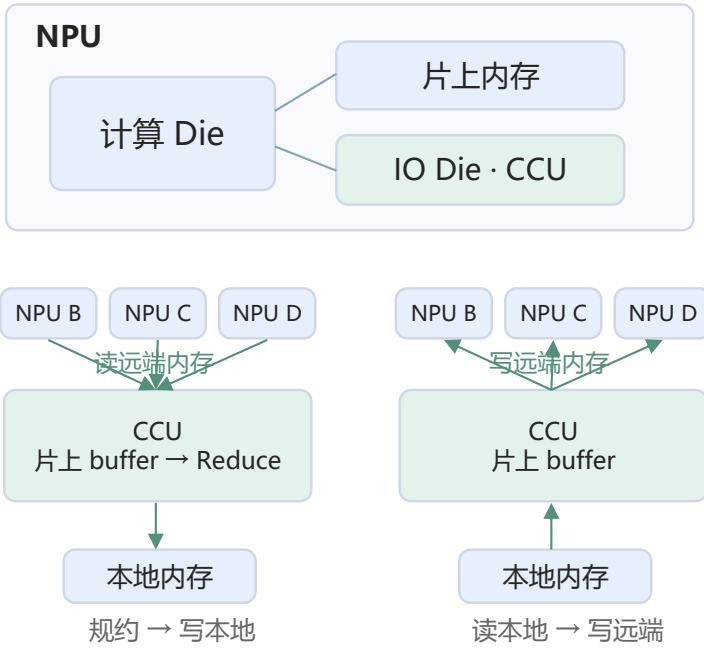


MoE 尾部：共享专家 ReduceScatter 保留 CCU



Ascend 950 专用通信引擎 CCU

CCU（集合通信单元）位于 IO Die，执行通信同步、数据传输与规约，减少对计算核的占用。



权重布局预处理

加载阶段将权重转为 NZ 格式，
减少运行时格式转换。

吸收转置，减少搬运

Value MatMul 直接输出 AllToAll 所需布局，
省去显式转置搬运；消除 Prolog 冗余 TensorMove。

实际 Decode Shape 调优

FA 优化贡献约 1 ms TPOT 收益
Prolog 优化贡献约 0.42 ms TPOT 收益
Gate BF16 优化贡献约 0.2 ms TPOT 收益

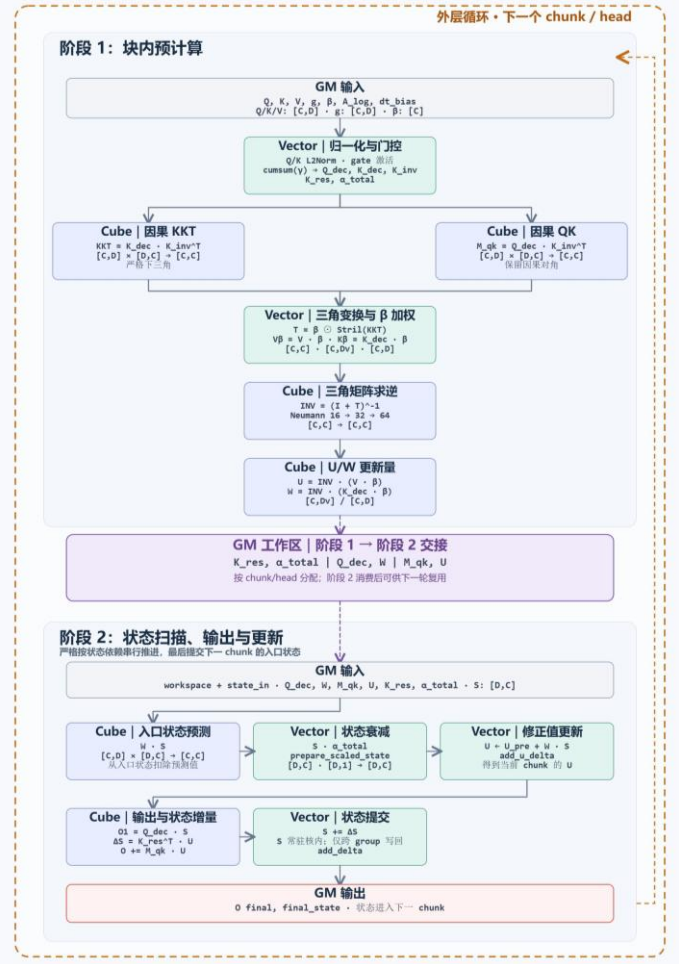
融合Kernel：FlashKDA

高性能FlashKDA

- **针对prefill场景，基于 CANNBot-DSL 快速构建全阶段极致性能大融合算子：**

- FlashKDA：高效实现的Prefill阶段融合算子，一次完成前处理、分块计算与跨 Chunk 状态递推。

分阶段+分组计算



高效矩阵求逆

矩阵分解

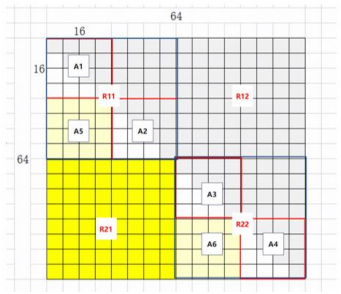
纯CUBE运算+巧妙布局L0C累加

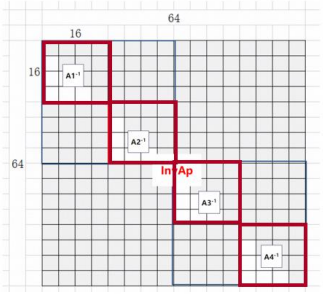
通过矩阵切分、小矩阵打包求解、L0C持久化累加中间结果加速小尺寸矩阵乘运算

STEP1

16×16小矩阵：Neumman级数倍增

$$(I-L)^{-1} = I + L + L^2 + \cdots + L^{C-1}$$



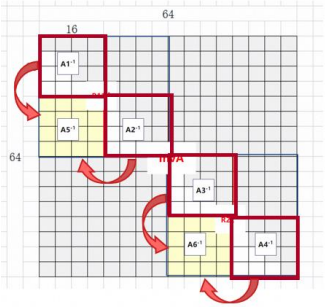


STEP2

32×32中矩阵：小矩阵分块迭代

$$INV = \begin{bmatrix} INV_{UL} & 0 \\ INV_{LL} & INV_{LR} \end{bmatrix},\quad L = \begin{bmatrix} L_{UL} & 0 \\ L_{LL} & L_{LR} \end{bmatrix}$$

$$INV_{LL} = -INV_{LR} \times L_{LL} \times INV_{UL}$$

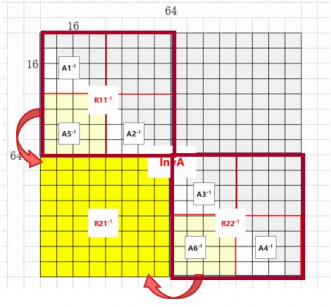


STEP3

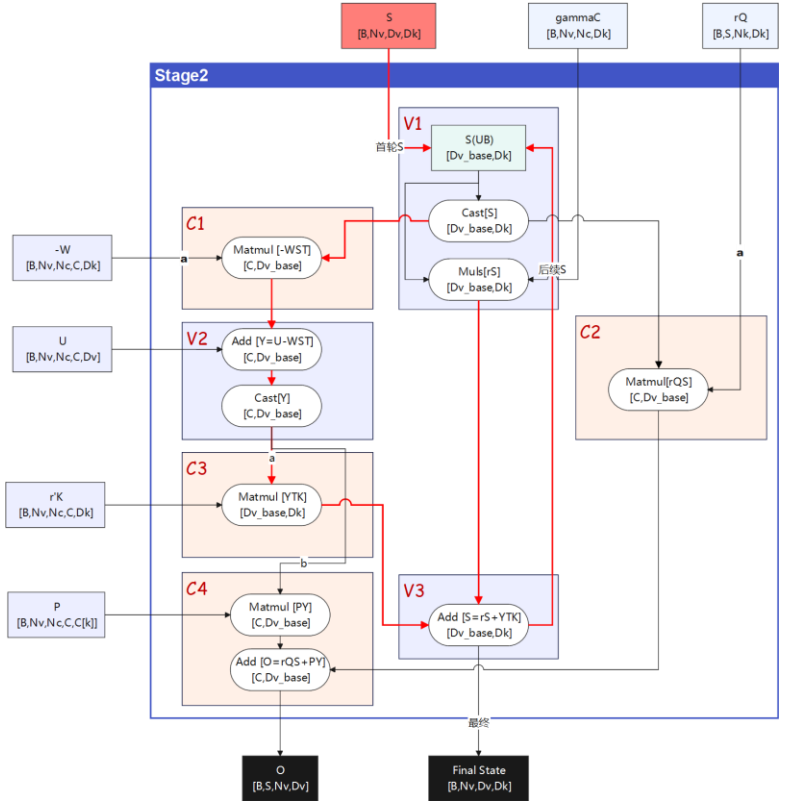
64×64大矩阵：中矩阵迭代

$$INV = \begin{bmatrix} INV_{UL} & 0 \\ INV_{LL} & INV_{LR} \end{bmatrix},\quad L = \begin{bmatrix} L_{UL} & 0 \\ L_{LL} & L_{LR} \end{bmatrix}$$

$$INV_{LL} = -INV_{LR} \times L_{LL} \times INV_{UL}$$



流水排布+负载均衡

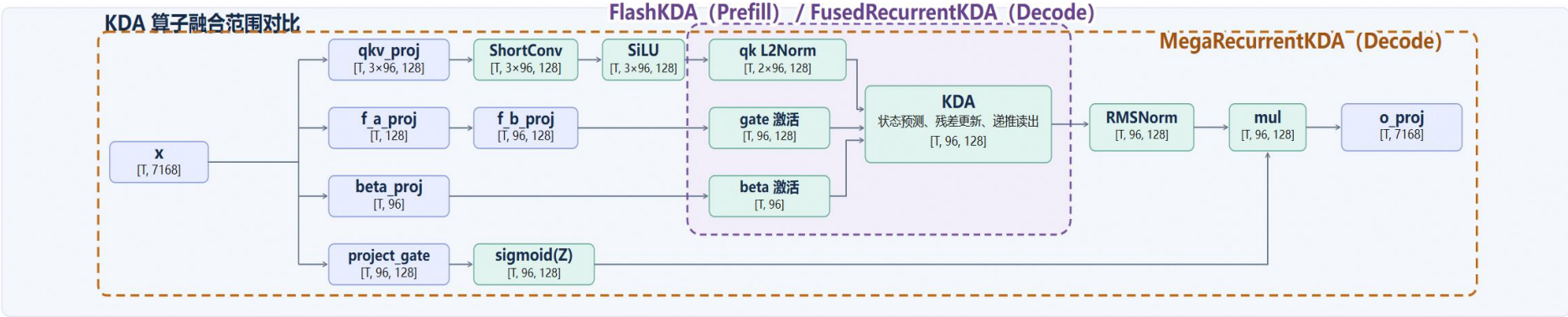


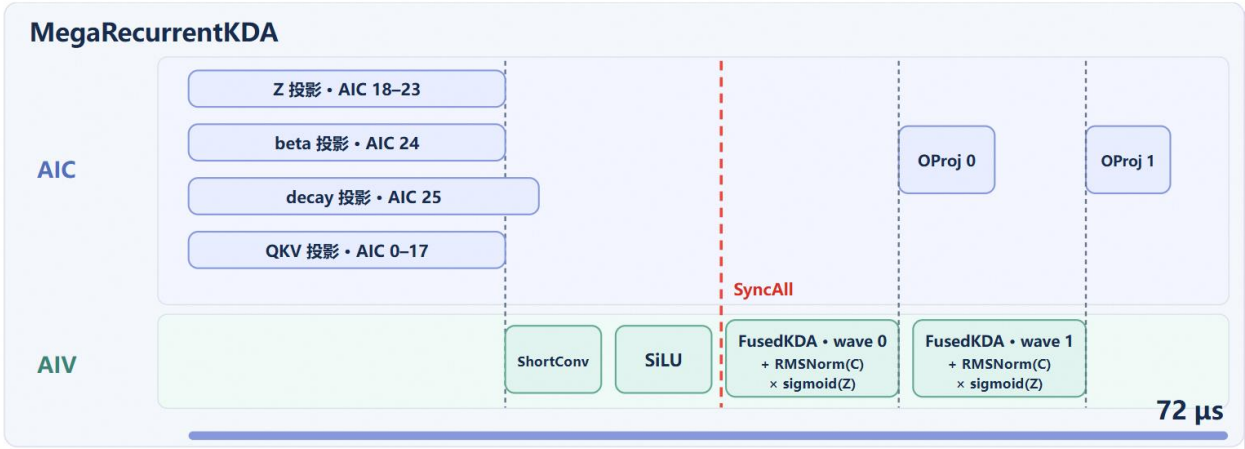
融合Kernel：MegaKDA

从FusedKDA到MegaKDA

➢ **针对decode场景，基于 CANNBot-DSL 快速构建KDA整层的大融合算子：**

- FusedRecurrentKDA：高效实现的Decode阶段融合算子，减少 Kernel 启动及 Global Memory 访存开销，提升逐 Token 解码效率；
- MegaRecurrentKDA：在FusedRecurrentKDA基础上进一步整层深度融合与流水线延迟隐藏，压缩访存及计算开销，释放 Decode 阶段极致性能;
- MegaRecurrentKDAReplaySSM/CommitRecurrentKDAReplaySSM：基于已接受前缀精准回放，减少冗余状态重缓存，减少投机验证带来的缓存压力。





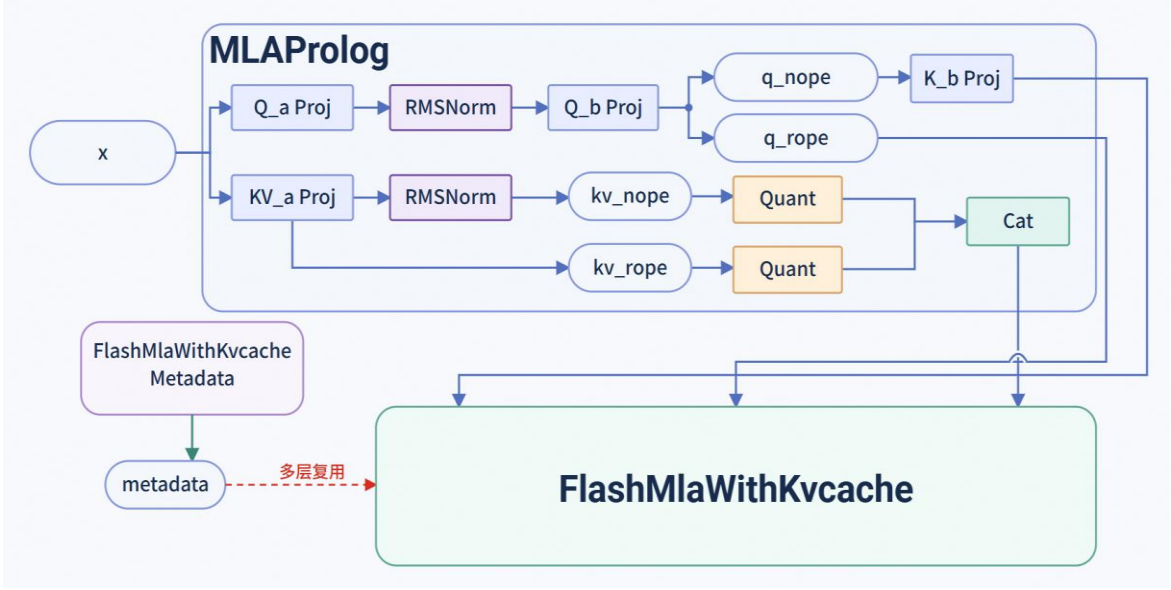


融合Kernel-其他结构

MLA结构

➢ **针对新的MLA模块，开发高性能融合算子：**

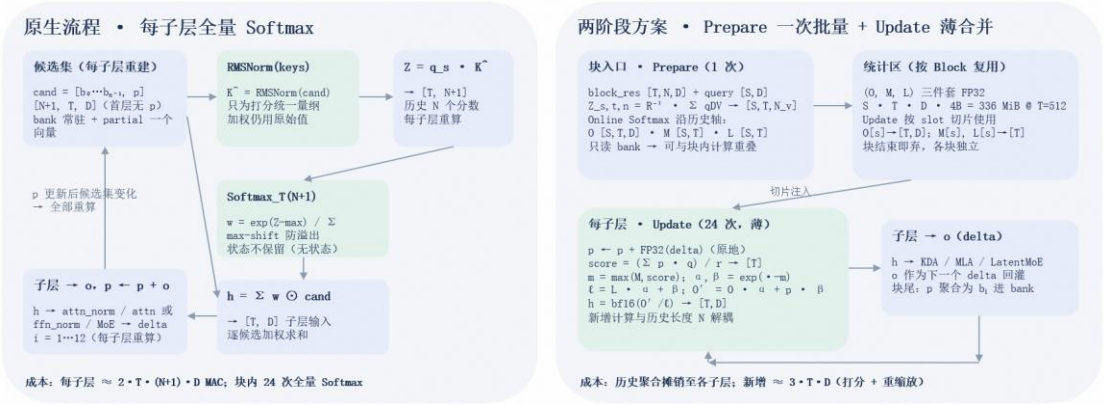
- FlashMlaMetadata/FlashMla：
  - 新增 AICPU算子实现Tiling结果计算和复用，降低Host侧开销;
  - 通过精细化的内存排布与访存优化等加速MLA计算, MFU可达90%;
- MlaProlog：新增去RoPE场景的MLA前置计算融合, 完成极致融合;



Block AttnRes结构

➢ **实现BlockAttnRes两阶段方案落地并高效极致性能优化**

- **两阶段方案设计**:使用AscendC开发BlockAttnResPrepare和Update融合算子，消除历史Block冗余计算，完成模型层面通用优化;
- **深度性能调优**:基于CANNBot-DSL推进算子纵向融合,实现Update与RMSNorm的进一步融合，进一步提升端到端推理效率;



MoE结构

➢ **新增 Situ 激活算法支持，扩展算子库**

- SituAndMulSparse：新增Situ算子, 扩展激活算法库;
- GroupSituMxquant：支持路由专家稀疏场景下的量化计算;

MLA 与 MoE 量化方案

MLA W8A8C8 + latent MoE W8A8

合计收益：约 1.7 ms / Token

MLA：展开 Prolog 计算

输入 X（BF16）→ 动态 MXQuant → X / scale（MXFP8）

Q 压缩 · q_a_proj · MXFP8 W8A8	KV 压缩 · kv_a_proj · MXFP8 W8A8
↓	↓
RMSNorm → 动态 MXQuant	拆分：KV latent（512）+ K（64）
↓	↓
Q 升维 · q_b_proj · MXFP8 W8A8	KV latent 做 RMSNorm → 与 K 合并
↓	↓
Q 吸收计算 · W_k 为 BF16	量化为 FP8 → 写入 KV Cache

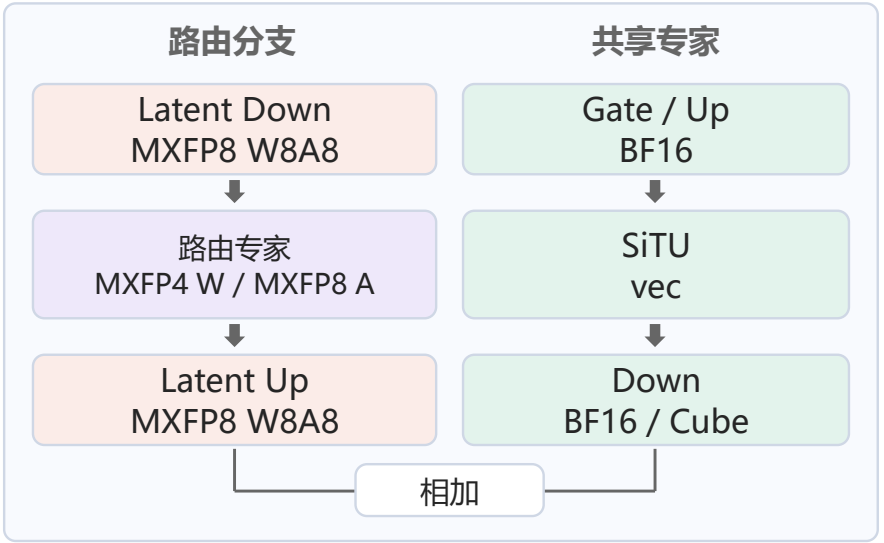
FP8 Q（576）+ descale

KV 数据容量约减半（相对 BF16）

FA：FP8 Q × FP8 KV Cache

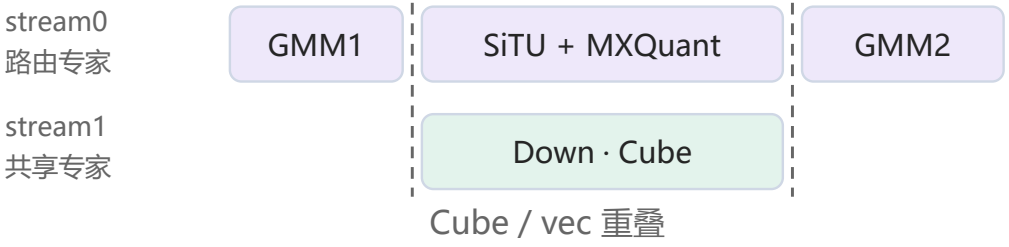
→ Value 投影 / Gate（BF16）→ o_proj（MXFP8 W8A8）

Stable LatentMoE

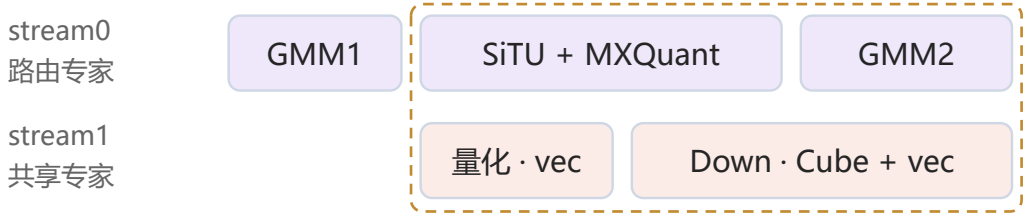


共享专家量化会影响哪一段并行?

当前：共享专家 BF16



若共享专家也量化



量化 + Down 均占用 vec，与路由分支争用

npugraph_ex：Pytorch图模式接入与编译优化

通过 torch.compile 接入 ACLGraph，在 PyTorch FX 图上叠加 NPU 亲和优化。

npugraph_ex：按场景组合优化特性

PyTorch 模型 → torch.compile → FX 图

↓

npugraph_ex 优化配置

FX 图优化 算子融合 / 冗余消除	**静态 Kernel 编译** 按 Shape 优化实现
模型编译缓存 编译结果落盘复用	**多流编排** 重叠计算与通信
计算资源控制 灵活分配 Cube / Vector	**SuperKernel** 融合 Kernel 调度

↓

CANN ACLGraph：图捕获与复跑

快速接入
ACLGraph 捕获与复跑
降低逐算子调度开销

高性能
FX 图叠加 NPU 优化
减少冗余计算与访存

可集成
服务化框架对接
vLLM / SGLang

配置驱动 • 按模型与业务场景组合优化能力

静态 Kernel 编译

面向固定或少量变化的 Shape

已知 Tensor 尺寸，优化存储与算子实现；
将 Scalar 计算前移到编译期，减少额外同步。

Shape 特化 → 降低运行开销

模型编译缓存优化

复用首次成图编译结果

将首次编译结果落盘，后续启动直接加载缓存；
减少 Dynamo 与 FX 图处理的重复编译耗时。

首次编译 → 落盘 → 后续复用

SuperKernel

基于图依赖的 Kernel 融合

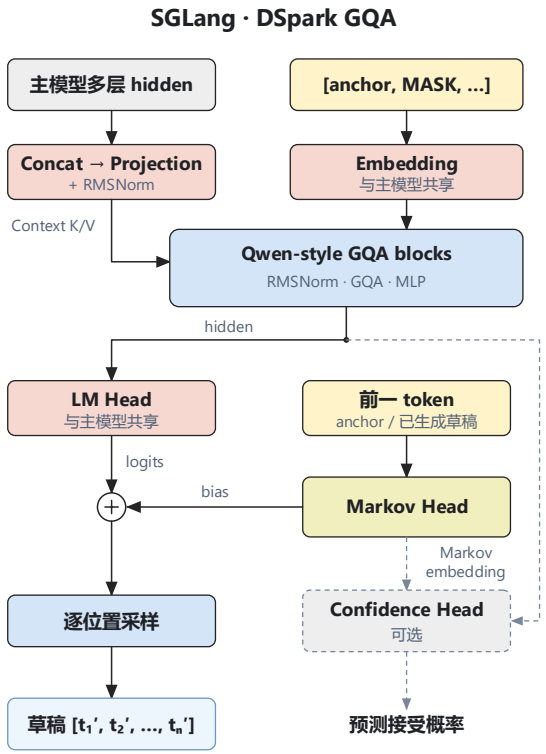


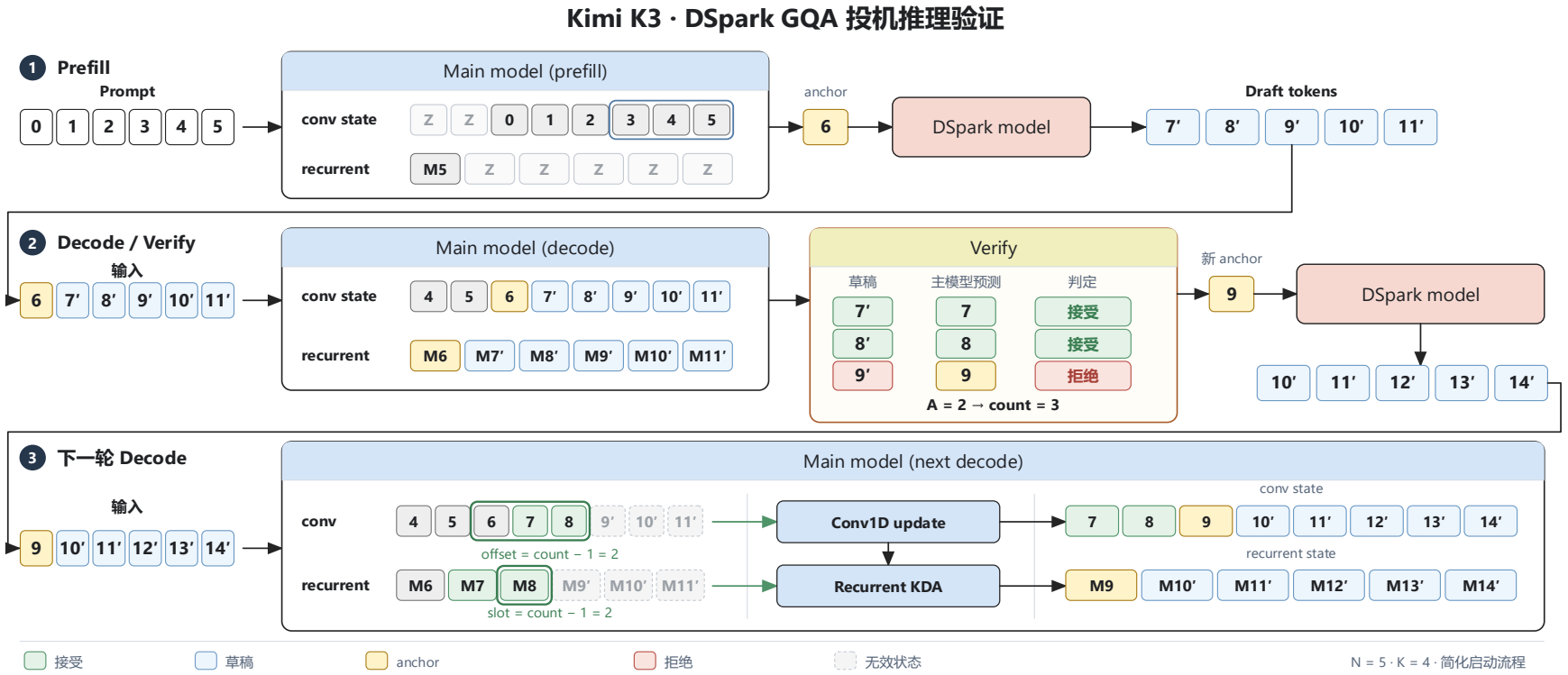
融合调度 → 降低 Kernel 间开销

Dspark: 投机推理优化

投机推理场景适配

为进一步优化 Kimi K3 Decode 阶段的 TPOT，当前基于 Hugging Face 开源的 RadixArk/Kimi-K3-DSpark 完成 Dspark 相关适配。通过草稿模型生成候选 Token、主模型批量验证并维护两套模型的状态 Cache，减少主模型逐 Token 执行次数，提升长文本和 Agent 场景下的生成效率。





投机投机验证的状态缓存开销与 ReplaySSM 优化

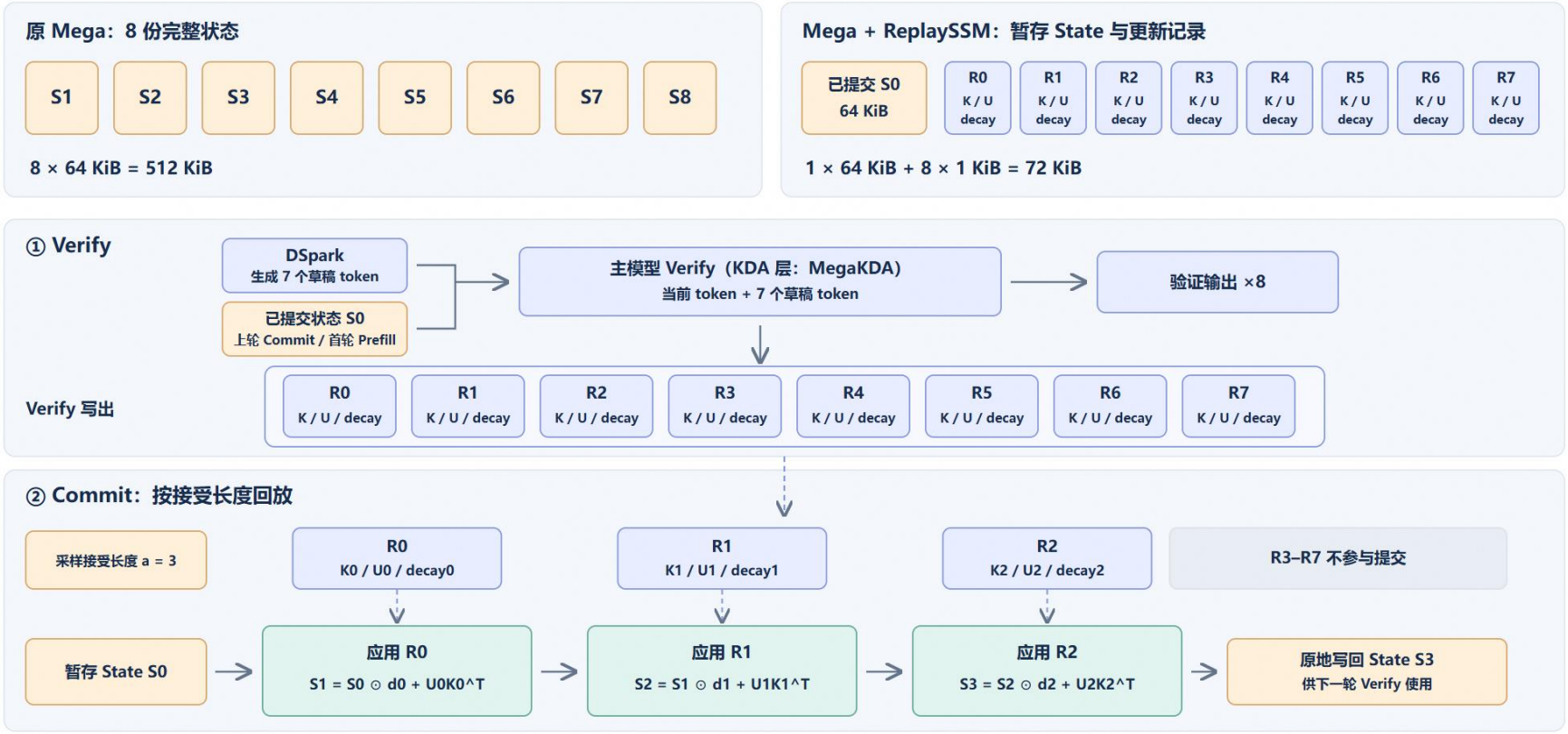
在 32 ranks/TP16/batch 256/next_n=7 的配置下，投机验证需为每个请求保存 8 份 recurrent state、10 个槽位的 conv state 快照，使 Mamba cache 从 3.374 增至 26.282 GiB/rank左右。面对recurrent/conv state的snapshot，我们需要引入业界的ReplaySSM方法来解决内存占用问题

投机推理访存进一步优化 —— ReplaySSM

Main model适配

- **ReplaySSM：减少投机验证的状态暂存**

在投机验证中，主模型一次处理当前 token 和 7 个草稿 token，共 8 个位置。传统方案需要为每个位置保存一份完整 KDA 状态，单个 Batch/Head 需要暂存 512 KiB。ReplaySSM 只保留本轮起点状态 S0，并记录 8 条轻量的 K/U/decay 更新记录；验证完成后，仅将采样器接受的前缀更新提交到状态。因此，状态暂存降至 72 KiB，减少 440 KiB，约 86%。



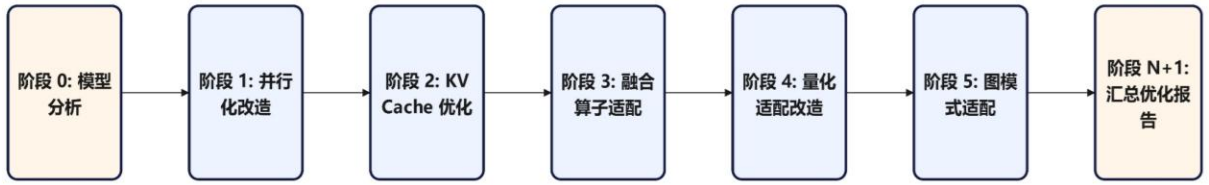
CANNBot支撑K3模型部署优化及性能提升

问题：通用 Agent 难以稳定完成长链路、高复杂度的 NPU 推理优化任务。
方案：模型部署涉及接口约束与硬件经验，口头传递容易遗漏。因此需要把步骤沉淀为Skill与知识库，并记录方案、实施和检查结果。
核心价值：开发效率提升、使用门槛降低。

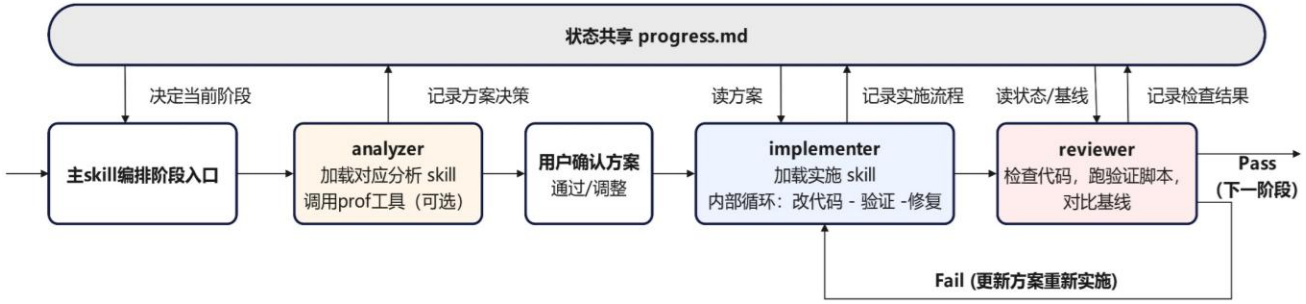
➢ Agent 端到端工作流 —— 稳定实现基础部署优化闭环

- 覆盖模型迁移适配、并行切分、KVCache 优化、融合算子替换、量化、图模式适配、debug 精度问题定位等 CANN 原子化优化特性，支持新模型一键式NPU部署优化。
- 通过完善的Harness 约束机制，包括 Agent team 角色分工、Progress 状态同步、Hook 约束和验证门禁，确保 Agent 稳定高效完成优化流程。

模型部署优化工作流



优化阶段内部流程



➢ 工作流驱动的人机协同 —— 进阶优化追求极致性能

- 通过profiling 采集与性能分析拆解 Skill，支持拆分模块/层耗时与抖动等性能瓶颈识别呈现，为优化项提供证据支撑。
- 利用多流、SuperKernel 等进阶优化 Skill， 使能优化特性进一步提升推理性能。

➢ 理论评估与性能拆解

- 整网理论性能评估 Skill，对接理论性能仿真引擎，在模型实际部署前，给出在不同负载、硬件抽象、部署策略等场景下的理论性能计算及显存分析，对后续部署策略选择和性能优化方案提供目标牵引。
- 算子性能拆解 Skill，在模型实际部署后，结合 Profiling实测，对比逐模块或逐算子性能与理论性能的差异，识别性能瓶颈与优化效果验证。

CANNBot驱动 Kimi-K3 核心算子极致效率开发和性能优化

充分应用 Agent 驱动热点模型核心算子开发，实现极致性能提升
- 中低难度算子：沿用 AscendC 开发范式，复用历史算子 skills 与知识库，快速完成算子开发和极致性能优化；
- 高复杂度 / Mega 大融合算子：基于 Agent Friendly 的 CANNBot-DSL 范式，Agent 主导开发，人机高效深度调优；

CANNBot 算子开发架构

Plugin 编排层
算子开发 · 性能调优

- **算子开发流程编排**
  AscendC · CANNBot-DSL· TileLang · Triton
- **性能调优流程编排**
  瓶颈分析 · 优化策略 · 回归验证

↓ 编排角色

角色层
角色分工 · 绑定技能

- `architect` 方案设计
- `developer` 代码开发
- `tester` 测试验证

↓ 绑定技能 · 按需调用

技能层
最小执行单元 · 按需调用

- **可复用 skills**
  编码规范 · 调优技巧 · 历史算子经验
- **知识库类**
  npu-arch · tiling · 编译器 · 调优技巧
- **调试类**
  精度定位 · 性能分析
- **辅助工具类**
  gitcode · cannsim

优化实践

中低难度算子&性能调优
AscendC 范式

复用参考历史算子 skills / 知识库

5 个算子

- `QuantFlashMlaWithKvcache(参考复用)`
- `SituAndMulSparse`
- `GroupSituMxquant`
- `BlockAttnResPrepare`
- `BlockAttnResUpdate`

→

高复杂度算子
CANNBot-DSL

Agent Friendly 范式，Agent 主导开发

4 个算子，覆盖KDA/Prolog等

- `MlaProlog`
- `FlashKDA`
- `CommitRecurrentKDAReplaySSM`
- `BlockAttnResUpdateRmsNorm`

→

Mega 大融合算子
CANNBot-DSL

多算子融合，Agent 主导开发与优化

2 个算子，KDA大融合算子

- `MegaRecurrentKDA`
- `MegaRecurrentKDAReplaySSM`

Takeaway

部署策略
Decode TP+DP+EP;
O-proj/Embedding/LM Head TP;

低比特量化
Weight W8A8 FP8/MXFP8；MoE W4A8 MXFP4 ； KV Cache伪量化C8；

融合Kernel
MoE: SituAndMulSparse, GroupSituMxquant,
MLA: QuantFlashMlaWithKvcache, MlaProlog
Block AttnRes: BlockAttnResUpdateRmsNorm, BlockAttnResPrepare, BlockAttnResUpdate
KDA: MegaRecurrentKDA, CommitRecurrentKDAReplaySSM, MegaRecurrentKDAReplaySSM, FlashKDA

通信优化
HCCL CCU/AIV通信;

多流并行与控核
MOE共享/路由专家多流，Mix算子控核，MLAProlog多流等;

Pytorch图模式：NpuGraph_Ex
static kernel + super kernel + compile cache优化

投机推理
dspark + replayssm

- Kimi K3 推理实践:https://gitcode.com/cann/cann-recipes-infer/tree/master/models/kimi_k3
- CANNBot-DSL:https://gitcode.com/cann/cannbot-dsl
- CANN transformer算子仓:https://gitcode.com/cann/ops-transformer
- CANNBot:https://gitcode.com/cann/cannbot

目录

Part 1 模型解析

Part 2 整网优化方案解析

Part 3 Future Plan

Future Plan

- **Prefill**：支持prefil CP部署
- **高吞吐性能优化**：进一步优化高吞吐场景性能
- **融合Kernel性能优化**：基于CANNBOT-DSL探索更大范围的MegaMoE 融合；
- **训练**：基于TorchTitan的训练支持；